A PROJECT REPORT

ExamIQ: A Smart Examination and Performance Analyzer

(Python Programming)

Desktop examination analytics built with Python 3 and Tkinter

Submitted By:

<<STUDENT NAME>>

UID: <<UID>>

Subject: Python Programming

Submitted To:

<<PYTHON SUPERVISOR>>

in partial fulfillment for the award of the degree of

MASTERS OF COMPUTER APPLICATIONS

IN

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY

October-November, 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "ExamIQ: A Smart Examination and Performance Analyzer" submitted by <<STUDENT NAME>> (<<UID>>), of University Institute Of Computing, Chandigarh University, is a Bonafide record of the work carried out under my guidance in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications at Chandigarh University during the academic year 2026-2028.

The project work embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

_____

Signature of Supervisor
<<PYTHON SUPERVISOR>>

Table of Contents

1. Abstract 4
2. Introduction 4
3. Objectives 5
4. Software & Hardware Requirements 6
5. Implementation 7
6. Key Functionalities 10
7. System Analysis & Differentiation 11
8. Flow Chart 13
9. Output & Screenshots 14
10. GitHub Upload 16
11. Conclusion 17
12. Future Advancements 17

Abstract

ExamIQ: A Smart Examination and Performance Analyzer is a Python 3 desktop application built with the Tkinter toolkit that replaces manual mark registers and hand-maintained result sheets with a single, consistent analytical workspace for a class of students. The application keeps every student record in a relational database through a thin, swappable adapter module (database.py), and derives from those rows the total marks, percentage, grade, rank, class average, pass rate and subject-wise averages of the whole class. All of this information is presented through four complementary visual forms: statistic cards for the headline numbers, bar charts that are drawn directly on a Tkinter Canvas, a sortable ttk.Treeview table for the student register, and a plain-text summary page that can be exported to a CSV file for archival or submission. The design deliberately demonstrates the three ways in which Python is normally used in an academic project: as a graphical client, as an integration layer, and as a data-access layer. The interface is composed from small reusable widget factories, the six screens are implemented as classes derived from tkinter.Frame and are switched by a single controller method, and each record can be handed to an external C++17 analytics engine through subprocess, whose JSON verdict is parsed back into Python dictionaries and rendered in the interface. A parallel Flask/web implementation (app.py with the templates folder) shares the same database module, which proves that the data layer was written independently of any one user interface. Input validation, parameterised SQL, transaction handling and a dual SQLite/Oracle back end complete the implementation without relying on any third-party desktop library.

Introduction

Result processing is one of the most repetitive tasks in an academic office. A faculty member has to copy marks from answer sheets into a register, add five subject marks for every student, convert the total into a percentage, apply a grading rule, sort the class to find the topper, and finally compute the average of each subject so that weak areas can be identified. When this is done by hand, or even in a general-purpose spreadsheet, the same arithmetic is repeated for every student, transcription errors are difficult to trace, and the analysis rarely goes beyond a single total column. ExamIQ was written to remove that repetition and to make the analysis visible.

The application is a native desktop program rather than a website, because a laboratory machine may not have a server, an internet connection, or permission to install one. Python and Tkinter satisfy this constraint completely: both ship with a standard CPython installation, so the project has no runtime dependency other than the interpreter itself. The same reasoning explains the choice of SQLite for the development database; the module database.py opens a single file (examiq.db) with the sqlite3 library, and it can be switched to an Oracle schema by setting the DB_MODE environment variable, without any change in the calling code.

The scope of the program is deliberately compact - five fixed subjects, one class, one row per student - so that the emphasis stays on the quality of the code. Every screen is a class, every repeated visual element is a factory function, every database statement is parameterised, and every computed figure is produced by a named function instead of being buried inside a button handler. These habits are what makes the small program a suitable demonstration of Python programming practice.

Objectives

The implementation was guided by the following objectives:

1. **Automate result computation:** remove manual addition and division entirely by computing the total, percentage, grade, class average, pass rate and subject averages inside Python functions from the stored marks.
2. **Build a maintainable graphical interface:** demonstrate object-oriented Tkinter programming in which every screen is a class derived from tkinter.Frame, styling is centralised in module-level constants, and repeated widgets are produced by reusable factory functions.
3. **Integrate Python with other languages:** call a compiled C++17 program through subprocess, exchange data as JSON, and parse the result back into Python dictionaries, so that Python is used as the orchestration layer of a mixed-language system.
4. **Protect data integrity:** validate every field before it reaches the database, use parameterised SQL everywhere, rely on constraints for the final guarantee, and keep all writes inside transactions.
5. **Produce usable reporting:** present analytics as cards, bar charts and sortable tables, and allow the ranking table to be exported to CSV so that results can be archived, reopened in a spreadsheet, or attached to a report.

Software & Hardware Requirements

1. Software Requirements

- **Programming Language:** Python 3.10 or later (developed and tested with CPython 3.14). Only the standard library is used by the desktop client.
- **Graphical Toolkit:** Tkinter / ttk with Tk 8.6, supplied with CPython; the clam ttk theme is re-styled to produce the dark interface.
- **Python Modules Used:** tkinter (widgets, ttk.Treeview, Canvas, filedialog, messagebox), sqlite3, csv, json, subprocess, os and sys. No pip package is required for the core application.
- **Database:** SQLite 3 (file examiq.db) for the demonstration mode; Oracle Database with the python-oracledb driver when DB_MODE=oracle is set.
- **External Engine:** C++17 compiler (g++ 16.2 from MSYS2 / MinGW-w64) to build cpp/analytics.cpp into analytics.exe, which the interface calls for the student-level analysis.
- **Optional Web Variant:** Flask 3.x for app.py, which exposes the same database module through a browser interface.
- **Development Environment:** Visual Studio Code, the Python extension, a PowerShell or MSYS2 terminal, and Git for version control.

2. Hardware Requirements

- **Processor:** 2.0 GHz dual-core Intel Core i3 / AMD Ryzen or better.
- **Memory:** 4 GB RAM minimum; the interpreter, the Tk event loop and SQLite together use under 150 MB.
- **Storage:** 500 MB free disk space for the interpreter, source files, the compiled engine and the database file.
- **Display:** 1366 x 768 or higher; the main window is designed for 1220 x 700 logical pixels and can be resized down to 1050 x 640.
- **Other:** Keyboard and mouse; no network connection is required at run time.

Implementation

The program follows the Model-View-Controller pattern. The three concerns live in separate files, so a change in one of them never forces a change in the other two. The Model is database.py, the View is tk_app.py with its page classes, and the Controller consists of the event handlers plus the ExamIQApp.show() routing method and the run_cpp() bridge to the compiled engine.

Model - database.py

The data layer hides every SQL statement behind eight functions: init_db, get_students, get_student, save_student, delete_student, get_summary, get_subject_averages and get_connection. Because the interface only ever calls these functions, the storage engine can be replaced without touching a single widget. A connection factory chooses the back end at run time, and sqlite3.Row is installed as the row factory so that columns can be read by name instead of by position.

Listing 1: the connection factory and the row-to-dictionary mapper

```
def get_connection():
    if DB_MODE == "oracle":                 # DB_MODE comes from the environment
        return _oracle_connection()         # python-oracledb
    conn = sqlite3.connect(DB_PATH)         # examiq.db next to the source files
    conn.row_factory = sqlite3.Row          # columns readable by name
    return conn

def _row_to_student(row):
    scores = [row["mathematics"], row["computer_science"], row["physics"],
              row["english"], row["dbms"]]
    result = dict(row)                      # plain dict for the interface
    result["scores"] = scores
    result["total"] = sum(scores)
    result["percentage"] = round(result["total"] / 5, 1)
    result["grade"] = ("A" if result["total"] >= 400 else
                       "B" if result["total"] >= 300 else
                       "C" if result["total"] >= 200 else "Needs Support")
    return result
```

The aggregate functions are deliberately written in Python rather than in SQL, because the same computations must work identically on SQLite and on Oracle. Dictionary comprehensions and generator expressions keep them short:

Listing 2: class-level statistics computed with max(), sum() and generator expressions

```
def get_summary():
    students = get_students()
    if not students:
        return {"students": 0, "class_average": 0, "top_score": 0,
                "top_name": "-", "pass_rate": 0, "needs_support": 0}
    top = max(students, key=lambda s: s["total"])
    return {
        "students": len(students),
```

```
        "class_average": round(sum(s["percentage"] for s in students) / len(students), 1),
        "top_score": top["total"], "top_name": top["name"],
        "pass_rate": round(sum(s["percentage"] >= 40 for s in students) * 100 / len(students)),
        "needs_support": sum(s["percentage"] < 40 for s in students),
    }
```

View - tk_app.py

The interface is a dark, sidebar-driven single-window application. Colours and fonts are declared once as module constants (BG, SIDEBAR, PANEL, BORDER, TEXT, MUTED, ACCENT, GREEN, VIOLET, AMBER, RED, FONT), so the visual identity of every screen comes from one place. Repeated elements are produced by four factories: make_button(), make_card(), make_panel() and make_table(); this removed roughly a hundred lines of duplicated configuration code and guarantees that all cards look the same.

Each screen is a class that inherits from tkinter.Frame, receives the application object in its constructor, builds its widgets once, and exposes a refresh() method that re-reads the database and updates the existing widgets instead of rebuilding them. That separation of construction and refresh is what makes switching between screens instantaneous.

Class	Responsibility
`ExamIQApp`	Main window: sidebar navigation, ttk style configuration, page registry and routing.
`DashboardPage`	Four statistic cards, subject-average chart, score-distribution chart, top performers.
`StudentsPage`	Treeview register with Add / Edit / Analyze / Delete buttons.
`EntryPage`	Add or edit form: student code, name, five subject marks, validation and save.
`RankingsPage`	Ranked table with rank number, total out of 500, percentage, grade and pass result.
`AnalysisPage`	Student selector, six result cards, subject breakdown bars, digit-frequency panel.
`ReportsPage`	Plain-text class summary and CSV export of the full ranking.

Navigation does not destroy and recreate screens. Every page is placed on the same parent container with place(), and show() raises the requested page with tkraise() after calling its refresh() method:

Listing 3: page registry, lazy placement and routing

```
class ExamIQApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("ExamIQ - Smart Examination & Performance Analyzer")
        self.geometry("1220x720")
        self.pages = {
            "dashboard": DashboardPage(self.content, self),
            "students":  StudentsPage(self.content, self),
            "entry":     EntryPage(self.content, self),
            "rankings":  RankingsPage(self.content, self),
            "analysis":  AnalysisPage(self.content, self),
```

```
            "reports":   ReportsPage(self.content, self),
        }
        for page in self.pages.values():
            page.place(in_=self.content, x=0, y=0, relwidth=1, relheight=1)
        self.show("dashboard")

    def show(self, key):
        for name, button in self.nav_buttons.items():     # highlight active item
            active = name == key
            button.config(bg="#0f2a3d" if active else SIDEBAR,
                          fg=ACCENT if active else MUTED)
        page = self.pages[key]
        page.refresh()                                    # reload data
        page.tkraise()                                    # bring to front
```

The two charts on the dashboard are not images. They are drawn with the Canvas widget from rectangles, lines and text items, and the drawing function is re-invoked on every Configure event so the chart always fills the window:

Listing 4: bar-chart rendering with Canvas primitives

```
def draw_bar_chart(canvas, labels, values, maximum, colors):
    canvas.delete("all")
    w, h = canvas.winfo_width(), canvas.winfo_height()
    left, right, top, bottom = 40, 15, 15, 38
    plot_h, plot_w = h - top - bottom, w - left - right
    for i in range(5):                                    # horizontal grid + y labels
        y = top + plot_h * i / 4
        canvas.create_line(left, y, w - right, y, fill=BORDER)
        canvas.create_text(left - 8, y, text=str(int(maximum - maximum * i / 4)),
                           fill=MUTED, anchor="e")
    slot = plot_w / max(len(values), 1)
    for i, (label, value) in enumerate(zip(labels, values)):
        x = left + slot * i + slot / 2
        bar_h = plot_h * (value / maximum) if maximum else 0
        canvas.create_rectangle(x - 18, top + plot_h - bar_h, x + 18, top + plot_h,
                                fill=colors[i % len(colors)], outline="")
        canvas.create_text(x, top + plot_h - bar_h - 9, text=str(value),
                           fill=TEXT, font=(FONT, 9, "bold"))
```

Controller - event handling and the C++ bridge

Validation happens before any SQL is executed. The entry form converts the five entries with int() inside a try/except, rejects empty fields, and refuses marks outside the 0-100 range; only then does it call save_student(). The same defensive style is used in the analysis screen, where the external program is launched with an explicit timeout and every possible failure (missing executable, non-zero exit status, malformed JSON) is converted into a message instead of an exception:

Listing 5: guarded subprocess call and JSON parsing

```
def run_cpp(student_code, scores):
    exe_path = os.path.join(BASE_DIR, "analytics.exe")
    if not os.path.exists(exe_path):
        return None, "analytics.exe not found. Compile it first (see README)."
    args = [exe_path, "analyze", str(student_code)] + [str(int(s)) for s in scores]
    try:
        result = subprocess.run(args, capture_output=True, text=True,
                                timeout=5, check=True)
        return json.loads(result.stdout), ""
```

```
    except (OSError, subprocess.SubprocessError, json.JSONDecodeError) as exc:
        return None, "C++ program failed: " + str(exc)
```

The Reports page demonstrates the standard library file-writing pattern: filedialog asks the user for a destination, the csv module writes a header row and then one row per student in ranking order, and the file is written with newline='' and encoding='utf-8' so that Excel and LibreOffice both open it correctly.

Figure: ExamIQ system architecture

Three layers: Python presentation, C++ computation engine, PL/SQL persistence

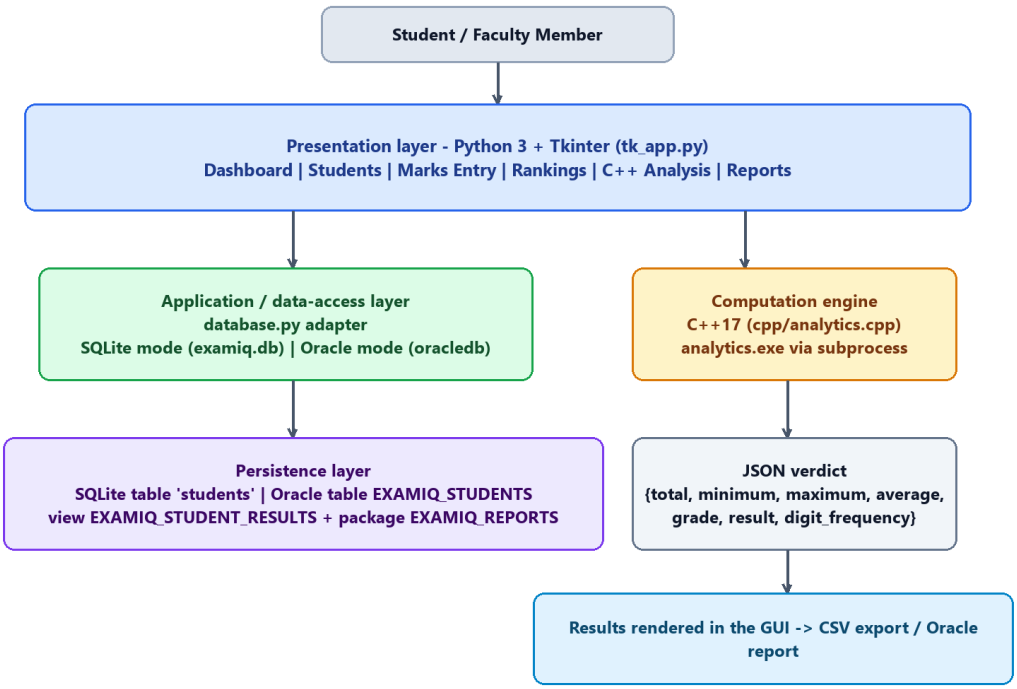


Figure 1: Modular architecture - Python presentation, C++ computation engine, SQL/PL-SQL persistence

Key Functionalities

1. Faculty Module (day-to-day use)

- **Dashboard:** four statistic cards (students, class average, top score with the topper's name, pass rate with the number of students below 40%), a subject-average chart, a five-bucket score-distribution chart and a live top-performers table.
- **Student register:** a ttk.Treeview listing every student with all five subject marks, total, percentage and grade, plus Add, Edit, Analyze and Delete actions bound to the selected row.

- **Marks entry and editing:** a two-column form with per-field labels, integer validation, range validation, a Clear button and an update path that reuses the same form for existing records.
- **Rankings:** students sorted by total marks with rank number, total out of 500, percentage, grade and PASS / FAIL result; sorting is done with sorted(students, key=lambda s: s['total'], reverse=True).
- **Reports and export:** a summary page reporting total students, class average, pass rate, highest and lowest total, strongest and weakest subject, and a one-click CSV export of the complete ranking.

2. Python Application Module (programming features)

- **Object-oriented screens:** six Frame subclasses, each with a constructor and a refresh() method, orchestrated by a single Tk subclass that owns the navigation state.
- **Reusable widget factories:** make_card, make_panel, make_button and make_table guarantee a consistent look and remove duplicated configuration code.
- **Canvas-based data visualisation:** both charts and the analysis progress bars are composed from create_rectangle / create_line / create_text items and are redrawn on resize.
- **Standard-library data handling:** csv for export, json to decode the C++ output, subprocess for process control, os/sys for portable paths, sqlite3 for persistence - no external dependency in the client.
- **Safe database access:** parameterised statements, context-managed connections that commit or roll back automatically, and executemany() for bulk seed data.
- **Graceful degradation:** if the compiled engine is absent the analysis screen shows an explanatory message instead of crashing, and the web variant of the project falls back to an equivalent Python calculation.

System Analysis & Differentiation

ExamIQ is intentionally small, so it was compared against the two tools that a college normally uses for the same job - a spreadsheet workbook and a general-purpose web-based student portal - as well as against typical academic mini projects.

- **Native desktop responsiveness:** screen changes are a tkraise() call, not a page load; the application runs on a laboratory machine with no server, no browser and no network.
- **Zero third-party dependency:** the entire client is standard-library Python, which removes the installation step that usually breaks classroom demonstrations, while the C++ engine remains an optional add-on.

- **One data layer, two interfaces:** database.py is shared by the Tkinter client and by the Flask variant in app.py, which demonstrates a genuine separation of concerns rather than two copies of the same query.
- **Portable persistence:** the same functions work against SQLite during development and against the Oracle schema in sql/oracle_setup.sql for the final demonstration.
- **Auditable computation:** every derived figure (total, percentage, grade, average, pass rate) is produced by a named function, so it can be inspected and tested independently of the interface.
- **Defensive input handling:** validation occurs in the form, again in the data layer, and finally in the database constraints; a non-numeric or out-of-range entry can never reach the table.

Flow Chart

The flowchart below traces a complete session: startup and database initialisation, dashboard aggregation, the marks-entry validation path, record persistence, ranking, the call into the C++ engine and the final report or export step.

Figure: ExamIQ process flow chart

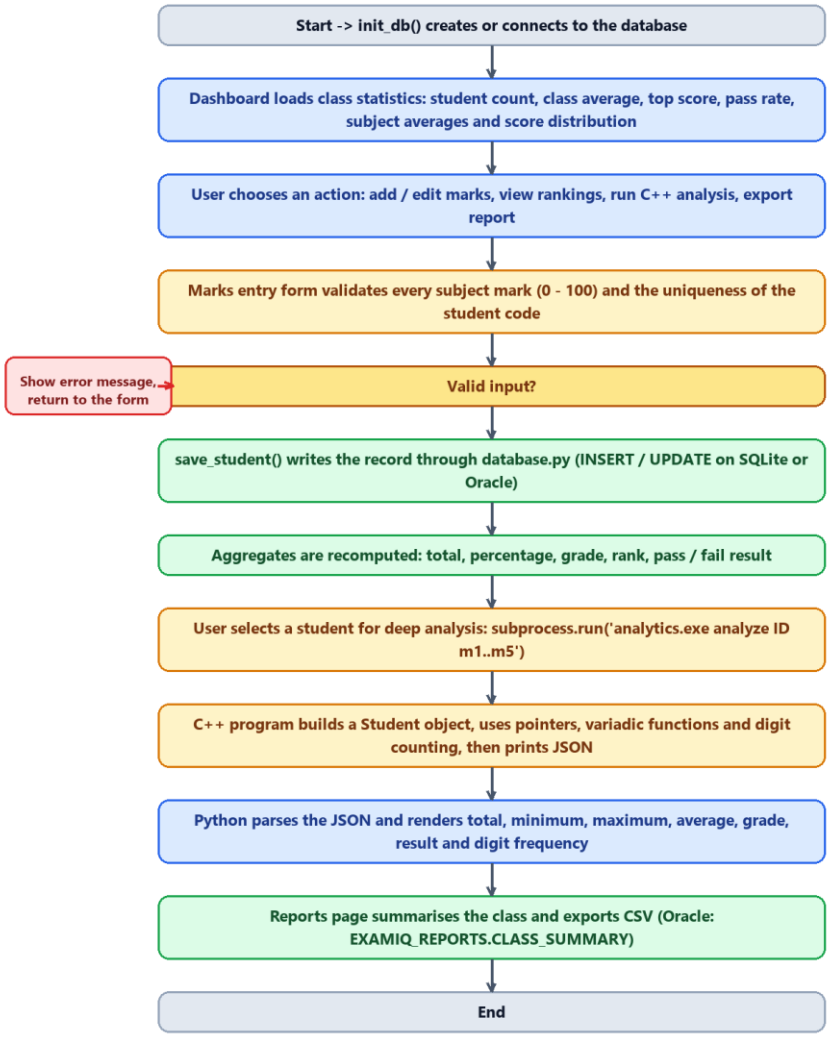


Figure 2: Process flow chart of the ExamIQ application

Output & Screenshots

The following screenshots were captured from the running application using the demonstration dataset of eight student records.

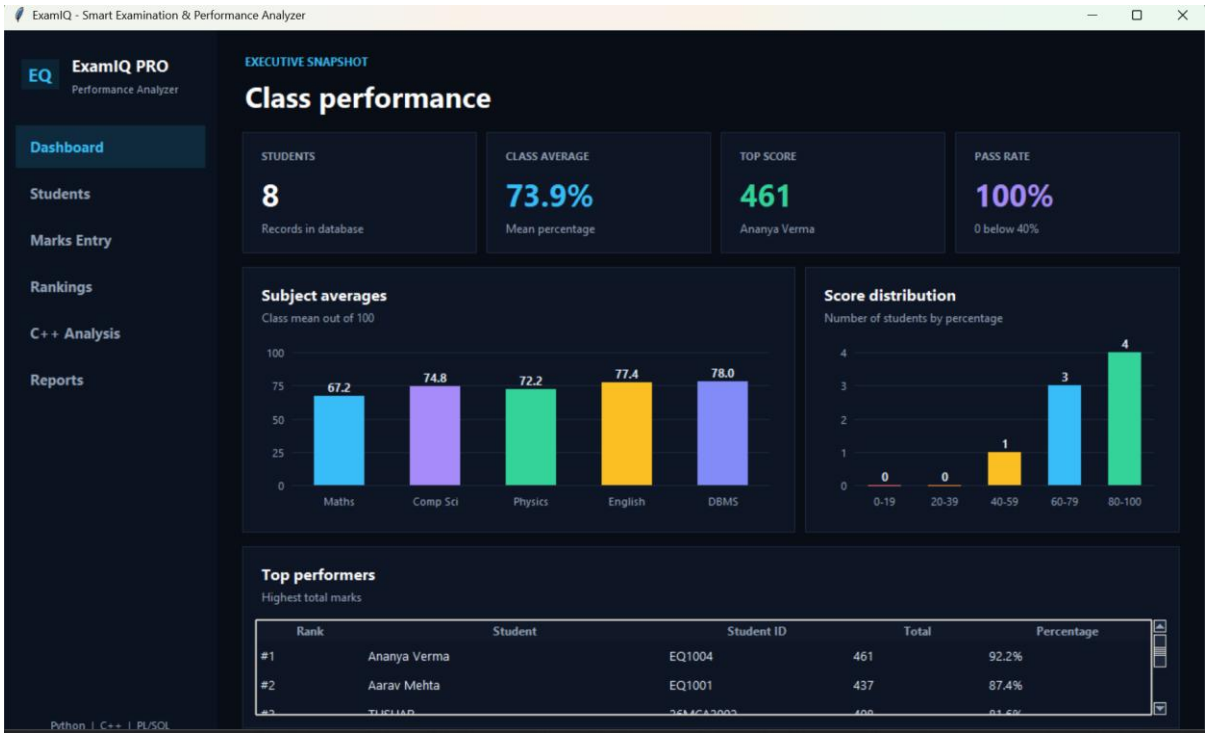


Figure 3: Executive dashboard - statistic cards, subject averages, score distribution and top performers

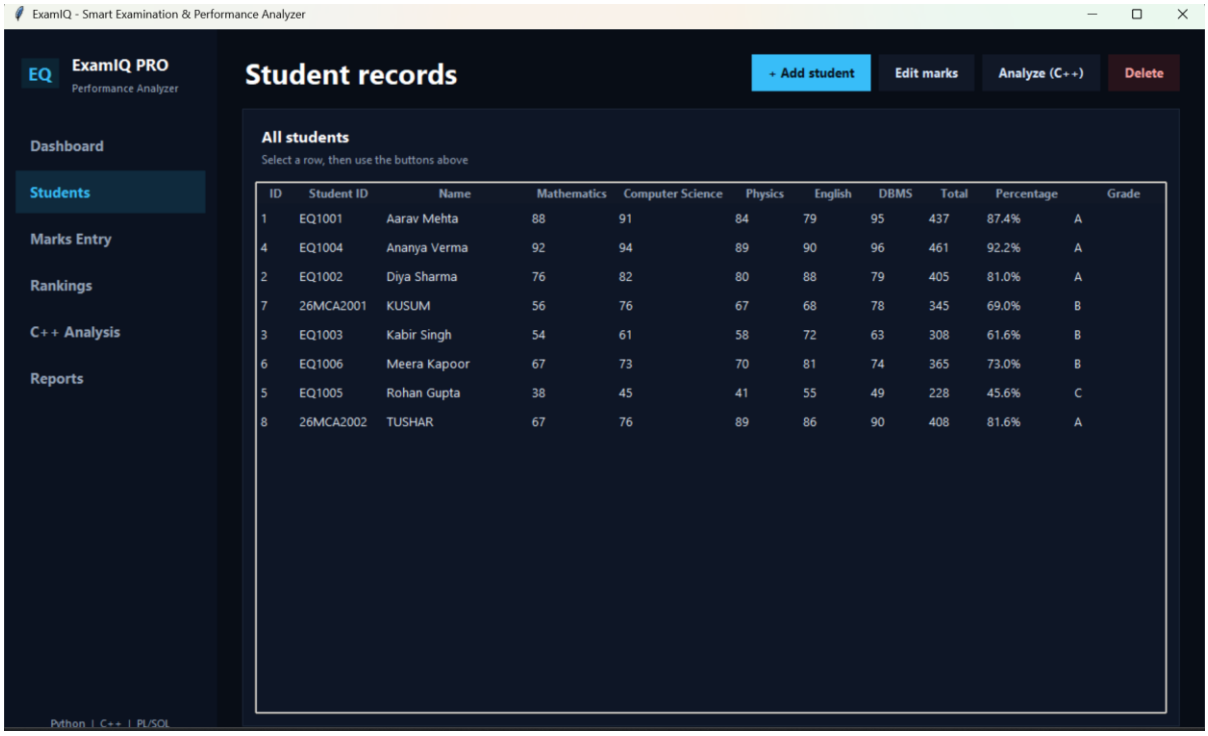


Figure 4: Student register - Treeview listing with totals, percentages and grades

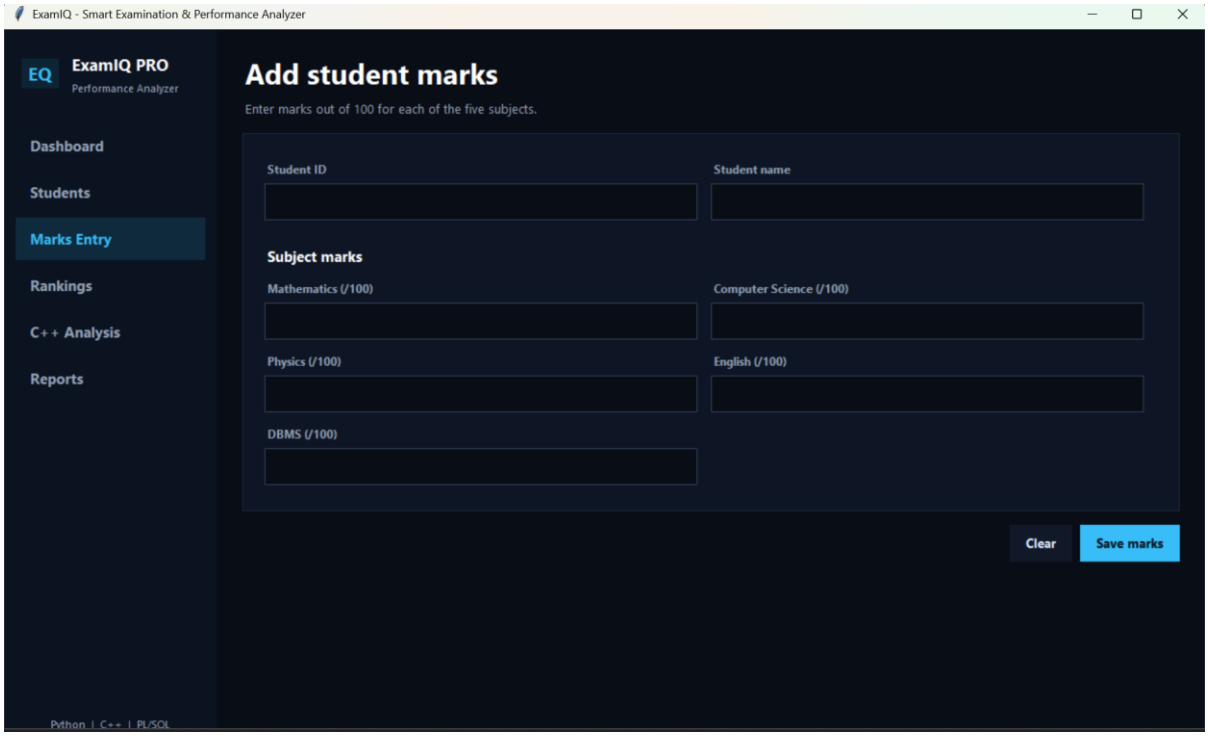


Figure 5: Marks entry form with per-subject validation

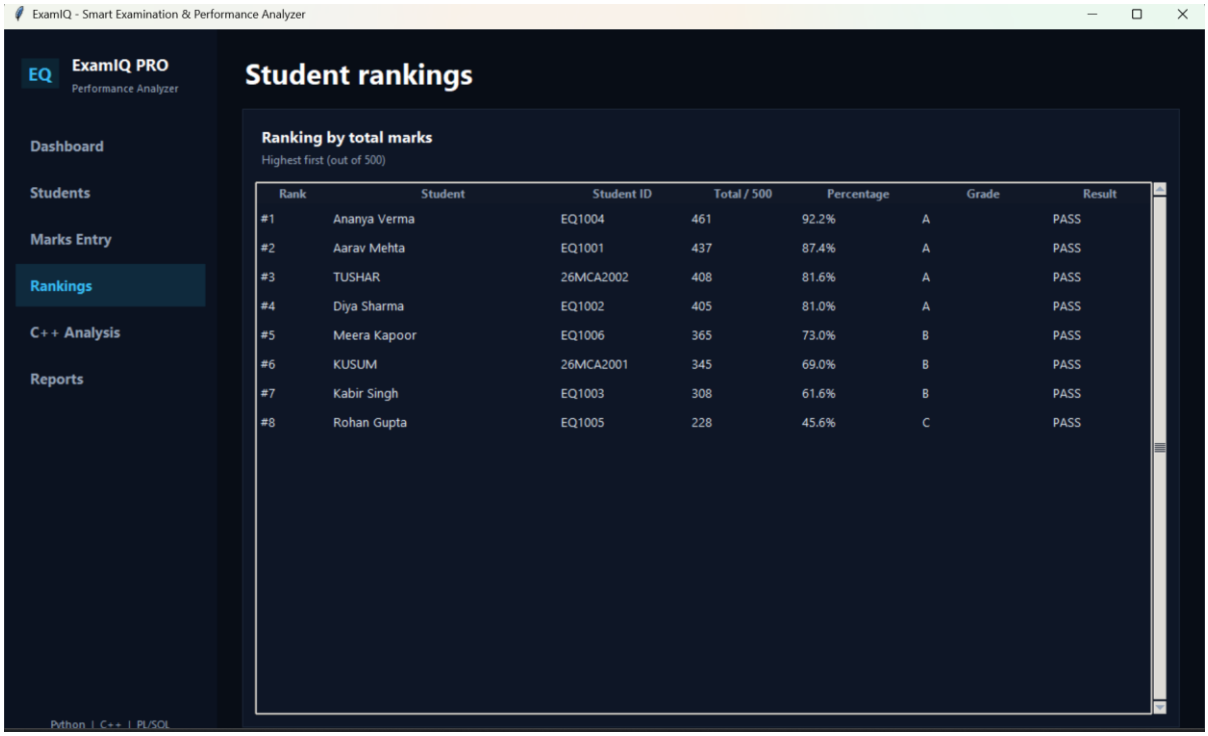


Figure 6: Ranking view - positions by total marks with grade and result

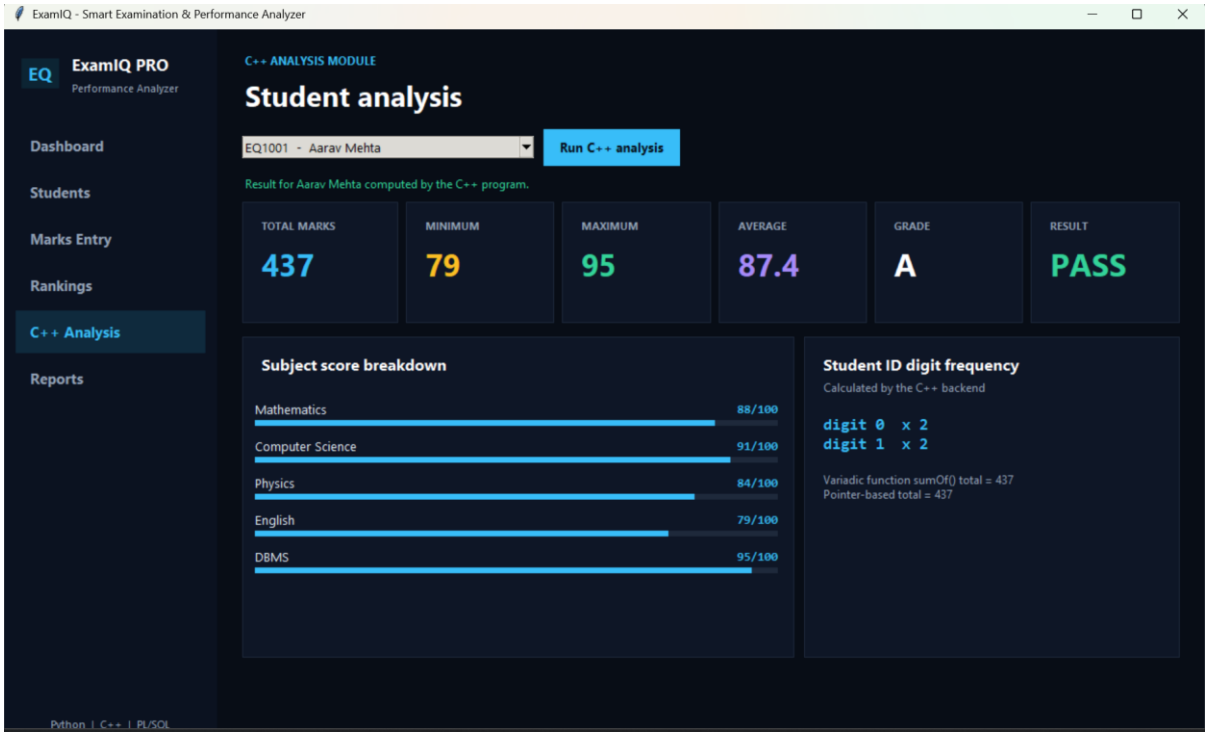


Figure 7: Student analysis computed by the C++17 engine and rendered by Python



Figure 8: Reports page - class summary ready for CSV export

GitHub Upload

The complete source code, the SQL scripts and the compiled analytics engine are maintained under version control and published in the following repository:

<<GITHUB REPOSITORY LINK>>

- tk_app.py, database.py and app.py - the Python client, the data layer and the web variant.
- cpp/analytics.cpp - the C++17 analysis engine with its build command.
- sql/oracle_setup.sql - the Oracle table, view and PL/SQL package.
- templates/ - the HTML templates of the Flask variant.
- README.md - setup instructions for both database modes and for the PL/SQL demonstration.

Conclusion

ExamIQ replaces a hand-maintained result register with a compact, dependency-free Python application that computes every examination statistic automatically and shows it in a form that a faculty member can read at a glance. The project demonstrates the practical use of object-oriented Tkinter programming, reusable widget factories, Canvas-based chart rendering, parameterised database access, transaction handling and standard-library file export. Because the data layer was written before the interface, the same code now serves both the desktop client and the Flask variant, and because the analytical engine was kept in a separate compiled program, Python could be used for exactly what it is best at: coordinating the user interaction and integrating the components of a mixed-language system.

The most valuable outcome of the exercise was not the arithmetic itself but the discipline that the arithmetic demanded - separating construction from refresh, validating before writing, and keeping every derived value behind a named function. Those decisions are what allow the program to be extended from a single class of five subjects to a full institution without being rewritten.

Future Advancements

- **Configurable subjects and weightings:** move the five fixed subject columns into a subjects table so that any programme, credit structure or internal-assessment split can be described by data rather than by code.
- **Multiple classes and semesters:** add class and term keys with a selection screen, which would allow historical comparison of the same cohort across examinations.

- **PDF result cards:** generate printable marksheets with reportlab or by driving Word/LibreOffice from Python, including the institution header, signatures and per-student grade cards.
- **Authentication and roles:** introduce a login screen with hashed passwords (hashlib.pbkdf2_hmac) so that marks can only be modified by authorised staff, and audit every change with a timestamp and user id.
- **Deeper analytics:** add subject-wise pass percentages, standard deviation, a z-score based difficulty index and a student-versus-class performance trend, all of which the current data model already supports.
- **Packaging and automated tests:** ship the client as a single executable with PyInstaller and cover the data layer with unittest cases that run against an in-memory SQLite database, so that the grading rules are verified on every change.